

Junior Developer take-home exercise

i Role: Junior Developer (AI focus), Engineering. **Timebox:** around 2 hours of focused work, and no more than 3. AI tools are encouraged.

Why we're doing this

We want to see how you build a small feature end to end: a simple API, a bit of data storage, and a frontend that talks to it. We care about clear, working code and your reasoning, not polish. You may (and should) use AI tools. We'll ask you to explain and tweak your solution during the interview, so build something you understand.

What to build

Brighte would like to collect expressions of interest for a new product, **Brighte Eats**, and view those leads in a simple list.

Customers can express interest in one or more services. **Service types:** delivery , pick-up , payment .

Core requirements

Backend (Node.js)

- A small API in JavaScript or TypeScript. A **REST** API is perfectly fine (GraphQL is welcome if you're comfortable, but not expected).
- **Create a lead:** an endpoint that accepts name , email , mobile , postcode , and services (one or more of the service types) and saves it.
- **List leads:** an endpoint that returns the saved leads. Letting the caller filter by service type is a nice touch but optional.
- **Storage:** your choice. A real database (SQLite, Postgres, etc.) is great, but a simple file or in-memory store is acceptable at this scope. Just tell us what you picked and why in the README.

Frontend (React)

- A **registration form** that submits to your create-lead endpoint, with basic validation, a loading state, and a success or error message.
- A **leads list** that fetches and displays the saved leads from your API.
- The frontend must talk to your **real API**, not mocked data.

General

- One or two meaningful tests (see below).
- A short **README** (template below).
- A **Git repo** we can check out and run easily.

A few decisions to make (and explain briefly)

We're not prescribing everything. Make sensible calls and note your reasoning in the README:

- **Validation:** what do you validate, and does it happen on the client, the server, or both?
- **Storage choice:** why did you pick it, and what would you use in a real product?
- **Basic safety:** no secrets committed to Git; avoid obvious input issues.

There's no single right answer. We mostly care that you can explain your choices.

Optional stretch (only if you have time, pick one)

Breadth is not the goal. Pick at most one: filter the list by service type; a lead detail view; loading and empty states done nicely; or a small AI touch (e.g. an endpoint that uses an LLM to summarise a lead). Stretch goals are genuinely optional.

Testing

Include a small number of meaningful tests. We're not after full coverage, just evidence you know what's worth protecting. Good candidates:

- The create-lead endpoint saves and returns a lead on the happy path.
- Submitting the form with a missing required field shows a validation error.
- The list endpoint returns the leads that were created.

A few well-chosen tests beat a large suite of shallow ones.

AI usage policy

- ✓ You may and should use AI tools (Copilot, Cursor, Claude Code, ChatGPT, etc.). Using AI well is part of how we work, and part of what we're assessing, along with your judgement about its output.

Add a short **"AI Assistance"** section to your README covering:

- Where AI helped (e.g. scaffolding, writing the form, tests).
- One place where you checked, corrected, or rejected the AI's output, and why.

- Any limitation you ran into.

We're assessing your understanding and ownership, not whether you typed every line.

Deliverables checklist

- ☐ Git repository we can check out
- ☐ Run instructions (and an `.env.example` if you use env vars)
- ☐ Working registration form and leads list against your real API
- ☐ One or two meaningful tests
- ☐ README using the template below

README template

```
1 ## How to run
2 ## What I built
3 ## Why I chose [storage / framework / library]
4 ## Validation strategy (client vs server)
5 ## What I'd change or add with more time
6 ## TODOs / known gaps
7 ## AI Assistance
8
```

Evaluation (what we look for)

- **Working end-to-end flow:** form to API to storage to list.
 - **Clean, readable code** you'd be comfortable putting up for review.
 - **Sensible decisions** on validation and storage, explained plainly.
 - **Frontend basics:** validation feedback, loading and error states.
 - **Test quality:** well-chosen, not exhaustive.
 - **Thoughtful AI use:** reaching for AI to move faster while staying critical of its output.
 - **Communication:** a clear README and the ability to walk us through your choices.
-

Submission

Reply with a link to your repo and instructions to run it locally. If anything is hard to run, tell us what you'd do to simplify it. Aim for around 2 hours and keep it under 3. It's completely fine to leave TODOs; just call them out. We value clarity and ownership over volume.